

124. 二叉树中的最大路径和

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Hard	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 动态规划, 二叉树
- **Companies:** (Not Available)

路径被定义为一条从树中任意节点出发, 沿父节点-子节点连接, 达到任意节点的序列。同一个节点在一条路径序列中 **至多出现一次**。该路径 **至少包含一个节点**, 且不一定经过根节点。

路径和是路径中各节点值的总和。

给你一个二叉树的根节点 `root` , 返回其 **最大路径和**。

示例 1:

输入 : `root = [1,2,3]`

输出 : 6

解释 : 最优路径是 `2 -> 1 -> 3` , 路径和为 `2 + 1 + 3 = 6`

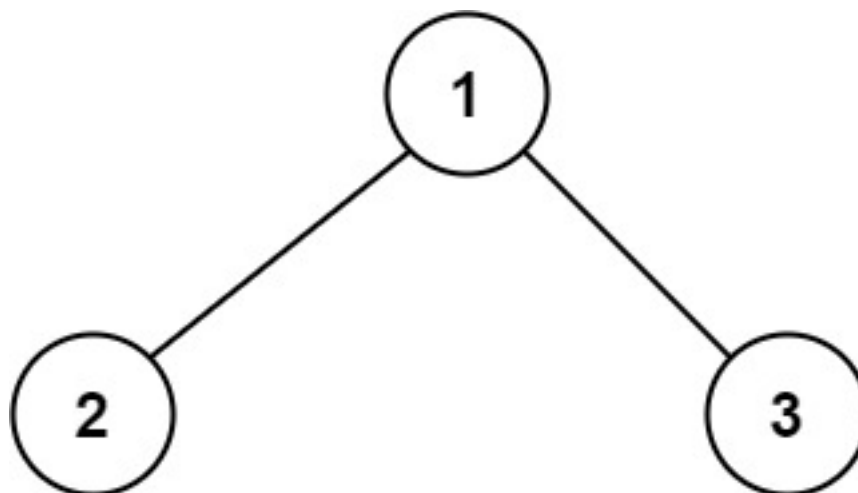


Figure 1: img

示例 2:

输入 : `root = [-10,9,20,null,null,15,7]`

输出 : 42

解释 : 最优路径是 `15 -> 20 -> 7` , 路径和为 `15 + 20 + 7 = 42`

提示:

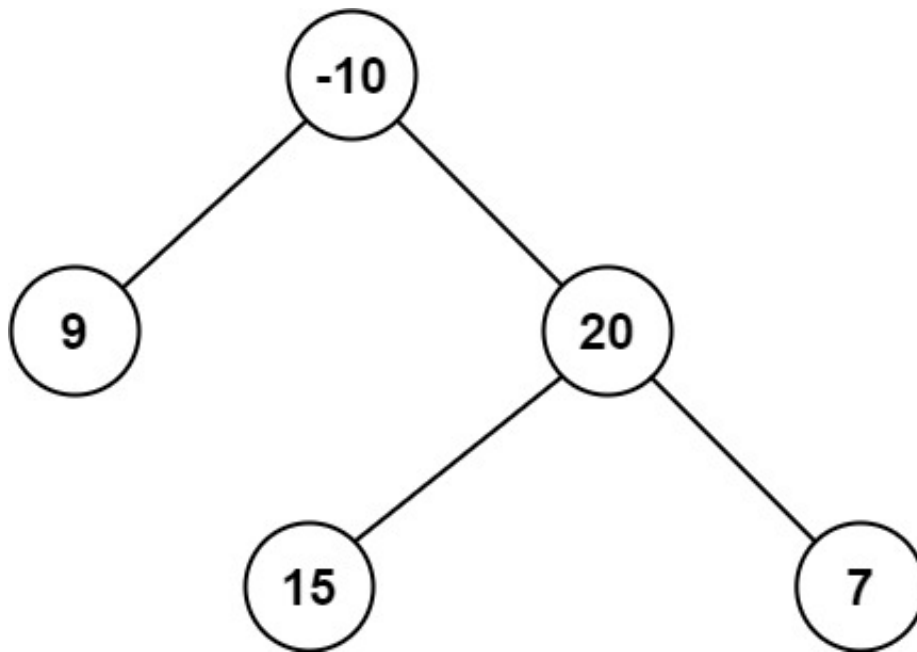


Figure 2: img

- 树中节点数目范围是 $[1, 3 * 10^4]$
- $-1000 \leq \text{Node.val} \leq 1000$

Discussion | Solution

解决方法

1. 问题理解

目标：找到二叉树中任意两个节点之间路径的最大和。**路径定义：** - 从任意节点开始，到任意节点结束。 - 只能沿着父子连接。 - 同一节点最多出现一次。 - 至少包含一个节点。 - 不一定经过根节点。

关键点： - 路径可以在任何节点开始和结束。 - 路径可能是一个“拐弯”的形状，即从左子树经过根节点到达右子树。这种路径不能再向上延伸。 - 路径也可能是一条从根节点向下延伸到某个子节点的“直路”。这种路径可以被父节点继续使用。 - 节点值可能为负数。如果子路径的和为负，我们宁愿不包含它。

2. 思路分析

递归（深度优先搜索 - DFS） + 后序遍历 对于任意一个节点 node，考虑经过它的路径：

1. 只包含 node 本身。
2. node + 左子树中的一条向下的最大路径。
3. node + 右子树中的一条向下的最大路径。
4. node + 左子树最大向下路径 + 右子树最大向下路径。（这种路径形成了一个“拐点”，不能再向上延伸）

我们需要计算每个节点作为“拐点”时的最大路径和，并维护一个全局最大值。同时，每个节点需要向其父节点返回一个信息：从该节点出发向下的最大路径和（只能走一边，左或右），以便父节点计算包含它的路径。

1. 定义递归函数 maxGain(node):

- 功能：计算从 node 出发向下的最大路径和（只能经过左子树或右子树或不经过子树），并在过程中更新全局最大路径和 max_sum。
- 返回值：从 node 出发向下的最大路径和 (node.val + max(左贡献, 右贡献))，如果这个值为负，则返回 0，表示不选择这条路径对父节点贡献更大。
- 基本情况 (Base Case): 如果 node 为 null，它不能贡献任何路径和，返回 0。
- 递归步骤 (Recursive Step):
 - 递归计算左子树提供的最大贡献: left_gain = max(maxGain(node.left), 0)。取 max(..., 0) 是因为如果子树贡献为负，我们宁愿不选它。
 - 递归计算右子树提供的最大贡献: right_gain = max(maxGain(node.right), 0)。
 - 计算以当前节点 node 为“拐点”的最大路径和: current_path_sum = node.val + left_gain + right_gain。
 - 更新全局最大路径和: max_sum = max(max_sum, current_path_sum)。
 - 计算并返回当前节点能向父节点贡献的最大路径和: 这个路径只能向下走一边（或者只包含当前节点）。return node.val + max(left_gain, right_gain)。

2. 主函数调用:

- 初始化全局最大路径和 max_sum 为一个极小值（例如 float('-inf') 或第一个节点的值）。
- 调用 maxGain(root) 开始递归计算。
- 最终返回 max_sum。

执行流程示例 (root = [-10,9,20,null,null,15,7]):

1. max_sum = -inf
2. maxGain(-10)
3. -> left = maxGain(9)
4. -> left_sub = maxGain(null) -> 返回 0
5. -> right_sub = maxGain(null) -> 返回 0
6. -> 节点 9: current_path_sum = 9 + 0 + 0 = 9. max_sum = max(-inf, 9) = 9. 返回 9 + max(0, 0) = 9. (left = max(9, 0) = 9)
7. -> right = maxGain(20)
8. -> left_sub = maxGain(15)
9. -> left_sub_sub = maxGain(null) -> 0
10. -> right_sub_sub = maxGain(null) -> 0
11. -> 节点 15: current = 15+0+0=15. max_sum = max(9, 15)=15. 返回 15+max(0,0)=15. (left_sub = max(15, 0) = 15)

12. -> right_sub = maxGain(7)
 13. -> left_sub_sub = maxGain(null) -> 0
 14. -> right_sub_sub = maxGain(null) -> 0
 15. -> 节点 7: current = 7+0+0=7. max_sum = max(15, 7)=15. 返回 7+max(0,0)=7. (right_sub = max(7, 0) = 7)
 16. -> 节点 20: current = 20 + 15 + 7 = 42. max_sum = max(15, 42) = 42. 返回 20 + max(15, 7) = 20 + 15 = 35. (right = max(35, 0) = 35)
 17. 节点 -10: current = -10 + 9 + 35 = 34. max_sum = max(42, 34) = 42. 返回 -10 + max(9, 35) = -10 + 35 = 25.
 18. 最终返回 max_sum = 42.

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import Optional
import math # 或者 float('-inf')

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        """
        计算二叉树中的最大路径和
        """
        # 初始化全局最大和为一个极小值
        self.max_sum = float('-inf')

    def maxGain(node: Optional[TreeNode]) -> int:
        """
        计算从 node 出发向下的最大单边路径和, 并更新全局 max_sum
        返回: 从 node 出发向下的最大单边路径和 (如果为负则视为 0)
        """
        if not node:
            return 0 # 空节点贡献为 0
```

```

# 递归计算左右子节点提供的最大单边贡献
# 如果子树贡献为负，则不选取，视为 0
left_gain = max(maxGain(node.left), 0)
right_gain = max(maxGain(node.right), 0)

# 计算以当前 node 为“拐点”的路径和，更新全局最大值
# 这个路径和 = 当前节点值 + 左子树最大贡献 + 右子树最大贡献
current_path_sum = node.val + left_gain + right_gain
self.max_sum = max(self.max_sum, current_path_sum)

# 返回当前节点能为父节点提供的最大单边贡献
# 这个贡献 = 当前节点值 + 左右子树中较大的那个单边贡献
# 注意：这个返回值不一定是全局最大路径和，它只是向上传递的信息
return node.val + max(left_gain, right_gain)

# 从根节点开始递归
maxGain(root)

# 返回记录的全局最大路径和
return self.max_sum

```

4. 复杂度分析

- **时间复杂度：** $O(N)$ ，其中 N 是树中的节点数。每个节点被访问一次。
- **空间复杂度：** $O(H)$ ，其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下（链状树）为 $O(N)$ ，平均情况下（平衡树）为 $O(\log N)$ 。

5. 如何在面试中讲解

1. 明确路径定义与难点：

- “目标是找到树中任意节点到任意节点的路径最大和。路径不一定过根，且可以在任意节点‘拐弯’。”
- “难点在于路径的多样性，特别是‘拐弯’路径和‘直路’路径的处理。”

2. 核心思路（递归/DFS）：

- “考虑对于每个节点 $node$ ，它对最大路径和的贡献有两种方式：”
 - “1. 作为路径的‘拐点’（最高点）：路径和 = $node.val$ + 左子树最大向下贡献 + 右子树最大向下贡献。”
 - “2. 作为路径的一部分（非拐点）：它需要向父节点提供一个‘单边最大贡献’，即 $node.val + \max(\text{左子树最大向下贡献}, \text{右子树最大向下贡献})$ 。”
- “我们需要一个递归函数，它既能计算以当前节点为拐点的路径和（用于更新全局最大值），又能返回当前节点的单边最大贡献给父节点使用。”

3. 递归函数设计 $\text{maxGain}(node)$ ：

- “返回值： $node$ 的单边最大贡献（如果为负则返回 0，表示不选）。”
- “全局变量： max_sum 记录全局最大路径和，初始化为负无穷。”
- “Base Case: $node$ 为空，返回 0。”
- “递归计算 $\text{left_gain} = \max(\text{maxGain}(node.left), 0)$ 和 right_gain ”

- = max(maxGain(node.right), 0)。取 max(..., 0) 是关键, 因为负贡献不如不选。”
- “更新全局最大值: max_sum = max(max_sum, node.val + left_gain + right_gain)。”
 - “返回单边贡献: return node.val + max(left_gain, right_gain)。”
4. **主函数逻辑:**
- “初始化 max_sum = -inf。”
 - “调用 maxGain(root)。”
 - “返回 max_sum。”
5. **复杂度分析:**
- “时间 $O(N)$, 每个节点访问一次。”
 - “空间 $O(H)$, 递归栈深度。”
6. **关键点强调:**
- “区分‘以节点为拐点的路径和’(用于更新全局 max_sum) 和‘节点的单边最大贡献’(递归返回值, 用于父节点计算)。”
 - “处理负贡献: max(gain, 0)。”
 - “全局变量 max_sum 的更新时机是在计算完左右子树贡献之后。”